pfi

Python for Fixed Income

COURSE 8 · IMPLEMENTING AGENTIC AI

An Orchestrator and Specialists

WEEK 3 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: an orchestrator and specialists

Week 3: many agents, kept honest

Day 11	an orchestrator and specialists
Day 12	memory and state
Day 13	evals
Day 14	guardrails and cost control

Today

1. Why split: what every request carries
2. specialist and route_table
3. Handoffs against agents as tools
4. The allow-list, asserted
5. route_table, and the desk brief
6. Parallel tool requests, once
7. What the orchestrator costs
8. Excel SUMIF, Git, the AI check

Every model response in the notebook is a saved response, replayed. A live run can differ.

The rules for agents

On the synthetic book: numbers from tools, described

Allowed names

DV01 per 100 of face

position DV01 in dollars

the largest DTS contributions in the file

Never called

cheap, rich, or attractive

a trade idea or a recommendation

overweight or underweight

On every agent's output: text, a saved run, a brief

Allowed names

the agent returned

the saved run

the brief states

Never called

the agent's opinion or view

what the agent believes

the agent's advice

Why split: what every request carries

```
# characters of tool schemas each agent sends with every request
macro                    2,974 characters of tool schemas
credit analyst           1,481 characters of tool schemas
risk                     6,749 characters of tool schemas
PM                           2 characters of tool schemas
one agent, every tool   11,204 characters of tool schemas
```

Shorter requests

only its own tools

Fewer wrong tools

less to choose among

One job each

instructions for one job

20 tools in one agent; the covenant tool maps 11 sections of Article 4 for the credit analyst.

Four specialists, each from an allow-list

```
risk = agentkit.specialist("risk", RISK, RISK_INSTRUCTIONS, aclient, model, 11).clone(
    output_type=agentkit.DeskAnswer, handoff_description=DESCRIPTIONS["risk"])
```

```
macro           tools  6; answer DeskAnswer
credit analyst  tools  3; answer CreditAnswer
risk            tools 11; answer DeskAnswer
PM              tools  0; answer text
```

route_table(result): one row per step: the agent, and a tool request, result, handoff, or answer.

specialist raises when the tools differ from the list. 51 passed with the day's two tests.

Handoffs against agents as tools

A handoff passes the conversation

transfer_to_<name>: one tool per specialist

the specialist reads the whole conversation

its answer ends the run

the orchestrator does not speak again

An agent as a tool keeps control

one tool per specialist, an input it writes

the specialist starts a run of its own

its answer comes back as a tool result

the orchestrator asks several, then answers

```
position DV01 of the book: 306,996.49 dollars per bp; the 10 Yr on 2026-10-02: 5.28 percent
```

The question needs two specialists: the book's DV01 (risk) and the 10 Yr in the file (macro).

Every check passes; the yield is wrong

4.0

percent: yield_to_maturity at par

5.28

percent: the file's 10 Yr

```
Total position DV01 in dollars of the synthetic holdings file: 306,996.493904 usd; pandas 306,996.493904; the...
10 Yr Treasury par yield on 2026-10-02: 4.0 pct; pandas 5.280000; the same at 2 decimals: no
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 4 requests, 6,610 prompt and 249 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

A 4 percent bond priced at 100 yields 4 percent, whatever the curve says. The tool was asked the wrong question.

One specialist received the whole question, and it needed two. The keys caught it; the checks could not.

specialist and the allow-list, asserted

```
ValueError: risk's tools are ['dv01'], not the allow-list ['dv01_per_100']
```

```
macro           holds exactly its allow-list: 6 tools
credit analyst  holds exactly its allow-list: 3 tools
risk            holds exactly its allow-list: 11 tools
PM              holds exactly its allow-list: 0 tools
risk            offered in its saved requests: 11 tools, the allow-list exactly
macro           offered in its saved requests: 6 tools, the allow-list exactly
```

The orchestrator offered only handoffs and agent tools: it holds no data tool, so it fetches no number itself.

route_table: who handled which step

Handoffs: the specialist's steps are the run's

```
1 orchestrator handoff transfer_to_risk
2 orchestrator handoff result to risk
3 risk tool request portfolio_totals
4 risk tool result {"as_of": "2026-09-30", "bonds": 200, "dts_years_bp": 1064.114013, "duration_...
5 risk tool request yield_to_maturity
6 risk tool result {"tool_call_id": "call_ifpnXWdtxKgXkTYDvd1JEXut", "yield_pct": 4.0}
7 risk answer {"text":"As of 2026-09-30, the total position DV01 in dollars of the syntheti...
```

Agents as tools: the specialists run inside a tool

```
1 orchestrator tool request risk
2 orchestrator tool result {"figures": [{"name": "Total position DV01 in dollars", "tool_call_id"...
3 orchestrator tool request macro
4 orchestrator tool result {"figures": [{"name": "10 Yr Treasury par yield on 2026-10-02", "tool_...
5 orchestrator answer As of 2026-09-30, the total position DV01 in dollars of the synthetic holdi...
```

The route table says who handled each step; the request table, read from the exchanges, says who sent each request.

The desk brief: three specialists, then the PM

4	2	4	2	1
orchestrator	macro	credit analyst	risk	PM

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 13 requests, 17,559 prompt and 1,017 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

```
numbers in the brief that no tool result holds: []
the credit analyst quotes Section 4.12; the quote is in that section word for word: yes; the brief holds the ...
```

The failing check is the credit analyst's own: a figure 2.0 with unit to_1, under the key text.

The PM has no tool. Every number in the brief traces to the specialists' answers; it added nothing.

Parallel tool requests, once

```
sdk_settings refused: agents.model_settings.ModelSettings() got multiple values for keyword argument 'paralle...
parallel_tool_calls: desk False, parallel_desk True
```

2

tool requests in one reply

6

requests, against 7 off

4

replayed from the run before

`the log: orchestrator, risk, macro, risk, macro, orchestrator`

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 6 requests, 7,598 prompt and 368 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Live, the interleaving depends on reply times: two runs can log two orders. So the course keeps it off.

What the orchestrator costs

```
run                agents   requests     prompt completion    largest        usd
handoffs                2          4       6610        249       2173   0.003042
agents as tools         3          7       8254        352       2030   0.003865
parallel on             3          6       7598        368       2030   0.003628
one agent               1          3       8066        115       2882   0.003410
desk brief              5         13      17559       1017       2185   0.008651
```

```
numbers in the answer that no tool result holds: ['298,998.53']
sector_totals returned 10 sectors, which add up to 306,996.49 dollars; pandas: 306,996.49
```

Model gpt-4.1-mini-2025-04-14, temperature 0, seed 42; 3 requests, 8,066 prompt and 115 completion tokens; saved run recorded 2026-10-11; replayed (no key loaded).

Twenty tools: the one agent took sector_totals and added the sectors itself, short of the key.

The split costs requests, and each request is smaller. Here it bought the right total.

Excel: requests and tokens by agent with SUMIF

```
=SUMIF($B$2:$B$15,"orchestrator",D2:D15)      sum range E, F, G: tokens and dollars
```

orchestrator	10 requests, 7,588 prompt, 412 completion tokens
risk	9 requests, 17,880 prompt, 632 completion tokens
macro	6 requests, 7,112 prompt, 369 completion tokens
credit analyst	4 requests, 6,277 prompt, 353 completion tokens
PM	1 requests, 1,164 prompt, 220 completion tokens
one desk	3 requests, 8,066 prompt, 115 completion tokens
PM, typed pm	1 requests, 1,164 prompt, 220 completion tokens

```
pm              Excel 1 requests; pandas: no agent named 'pm'
```

Excel 16.0 build 20430, typed by script. SUMIF ignores case ("pm" finds PM); pandas' == does not.

Git: runs.csv by agent, and bring it home

```
agentkit day 11: specialist, route_table, 51 tests; runs.csv by agent
Start day 11: the earlier modules and agentkit as day 10 left it, 49 tests
agentkit.py      | 41 ++++++++++++++++++++++++++++++++++++++++++++
runs.csv         | 15 +++++++++++++++
test_agentkit.py | 24 ++++++++++++++++++++++++
3 files changed, 80 insertions(+)
```

```
python -m pytest -q test_agentkit.py          # 51 passed
git add agentkit.py test_agentkit.py
git commit -m "agentkit day 11: specialist, route_table"
```

One row per agent per run: git log -p runs.csv later shows which agent's numbers changed.

The AI check

The docstring is the prompt. The agent code that comes back runs and looks right.

1

Docstring

Exactly an allow-list: one job, its own tools, nothing else.

→

2

Code back

Written for the course in the style of an AI answer. It runs.

→

3

Test first

Build all four; assert each one's tools. The PM holds 20, dv01 among them.

→

4

Fix

The allow-list and nothing else, checked: 6, 3, 11, and 0 tools.

Today: specialist() hands every agent all 20 tools "so none gets stuck": the PM can compute a DV01 itself.

Code that runs is not the same as code that is right.

What you can do now

- Why a desk of agents splits the work
- specialist: tools equal to an allow-list, asserted
- Handoffs pass the conversation; tools keep control
- route_table and the request table, side by side
- A brief from an agent with no tools, checked

Week 3, Day 12 Memory and state